```
        return Forbid();
    }
```

In my version of the application, Paul is the user who has an `Id` starting with `beed`. As you can see from figure 10.8, he is the creator of the property with an `Id` of `1`. If I log in as Paul and navigate to /property-manager /edit/1, I have access to the Edit form. If I attempt to replace the 1 in the URL with a 2, I am presented with the Access Denied page because Paul is not the creator of the property with an `Id` of 2.

If you prefer to check against a policy, rather than the requirement directly, you would register the policy like this:

```
builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("EditPropertyPolicy", policyBuilder =>
        policyBuilder.AddRequirements(PropertyOperations.Edit));
});
```

Then you would pass the name of the policy in to the `AuthorizeUserAsync` method instead of the name of the requirement:

```
var result = await _authService.AuthorizeAsync(User, property,
➥ "EditPropertyPolicy");
```

As well as requiring a bit more work, checking against a policy entails the use of strings, which you can get around by using constants, but that would require even more work.

There is still one thing to be attended to. At the moment, the property manager Index page lists all properties with links to see their details, edit, and delete them. It seems pointless presenting a user with a link to edit or delete a property when they are not authorized to do so. Ideally, you would